

DEMAND ANALYSIS

Ali Can TURAN

Aspiring Supply Chain Data Scientist

08 / 2026

BOTH VIA MS EXCEL AND MACHINE LEARNING

<https://github.com/ali-can-turan/>

<https://github.com/ali-can-turan/case-studies-academic/tree/main/Demand%20Analysis>

ali.can.turan@outlook.com

ACTUAL DEMAND VALUES - TABLE

2

month year	10-Piece Set sold units	Time in month	10-Piece Set sale price	Feb	Mar	Apr	May	Jun	Jul	Aug	Oct	Nov	Dec
Jan-11	219	1	\$949.98	0	0	0	0	0	0	0	0	0	0
Feb-11	175	2	\$949.98	1	0	0	0	0	0	0	0	0	0
Mar-11	306	3	\$949.98	0	1	0	0	0	0	0	0	0	0
Apr-11	394	4	\$949.98	0	0	1	0	0	0	0	0	0	0
May-11	394	5	\$949.98	0	0	0	1	0	0	0	0	0	0
Jun-11	131	6	\$949.98	0	0	0	0	1	0	0	0	0	0
Jul-11	131	7	\$949.98	0	0	0	0	0	1	0	0	0	0
Aug-11	438	8	\$949.98	0	0	0	0	0	0	1	0	0	0
Sep-11	525	9	\$949.98	0	0	0	0	0	0	0	0	0	0
Oct-11	613	10	\$949.98	0	0	0	0	0	0	0	1	0	0
Nov-11	657	11	\$949.98	0	0	0	0	0	0	0	0	1	0
Dec-11	394	12	\$949.98	0	0	0	0	0	0	0	0	0	1
Jan-12	354	13	\$949.98	0	0	0	0	0	0	0	0	0	0
Feb-12	706	14	\$949.98	1	0	0	0	0	0	0	0	0	0
Mar-12	603	15	\$949.98	0	1	0	0	0	0	0	0	0	0
Apr-12	603	16	\$949.98	0	0	1	0	0	0	0	0	0	0
May-12	367	17	\$949.98	0	0	0	1	0	0	0	0	0	0
Jun-12	328	18	\$949.98	0	0	0	0	1	0	0	0	0	0
Jul-12	509	19	\$949.98	0	0	0	0	0	1	0	0	0	0
Aug-12	429	20	\$949.98	0	0	0	0	0	0	1	0	0	0
Sep-12	417	21	\$949.98	0	0	0	0	0	0	0	0	0	0
Oct-12	1,553	22	\$799.95	0	0	0	0	0	0	0	1	0	0
Nov-12	1,670	23	\$799.95	0	0	0	0	0	0	0	0	1	0
Dec-12	1,204	24	\$799.95	0	0	0	0	0	0	0	0	0	1

Continuous variables

- Time in month
- 10-Piece Set sale price

Categoric variables

- Months → N-1 rule applied: 2 months with baseline demand removed.

Target variable

- 10-Piece Set sold units

VIA EXCEL ANALYSIS TOOLPAK REGRESSION

VIA EXCEL ANALYSIS TOOLPAK REGRESSION

SUMMARY OUTPUT	
Regression Statistics	
Multiple R	0.974241697
R Square	0.949146885
Adjusted R Square	0.893670759
Standard Error	129.6407713
Observations	24

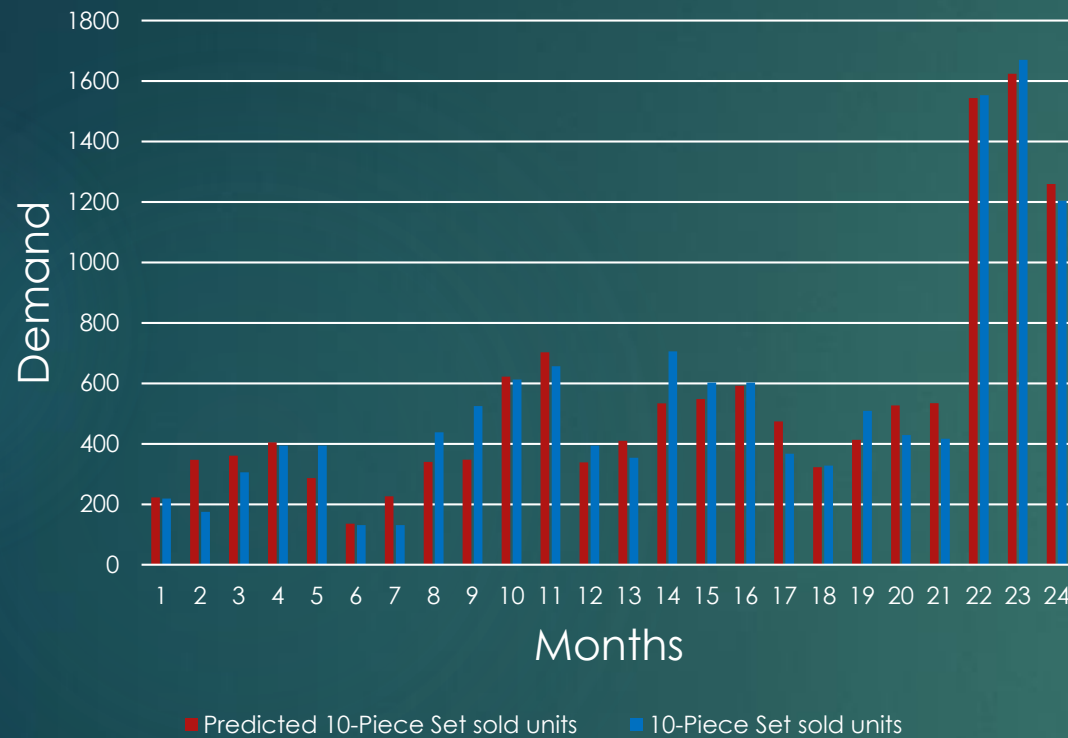
- Categorical variable 'Nov' is the only one with a significant p value.
- Train R square is 94.92%.
- Test R square is 42.93%.
 - Normal with scarce test data → test_size=3

	Coefficients	P-value
Intercept	4855.135495	7.66E-05
Time in month	15.58146067	0.008349
10-Piece Set sale price	-4.892504645	8.03E-05
Feb	108.494382	0.358496
Mar	106.9129213	0.363141
Apr	135.3314607	0.253762
May	1.75	0.987843
Jun	-164.8314607	0.170437
Jul	-89.91292135	0.441833
Aug	8.005617978	0.944896
Oct	259.3314607	0.060879
Nov	324.25	0.024156
Dec	-55.83146067	0.661821

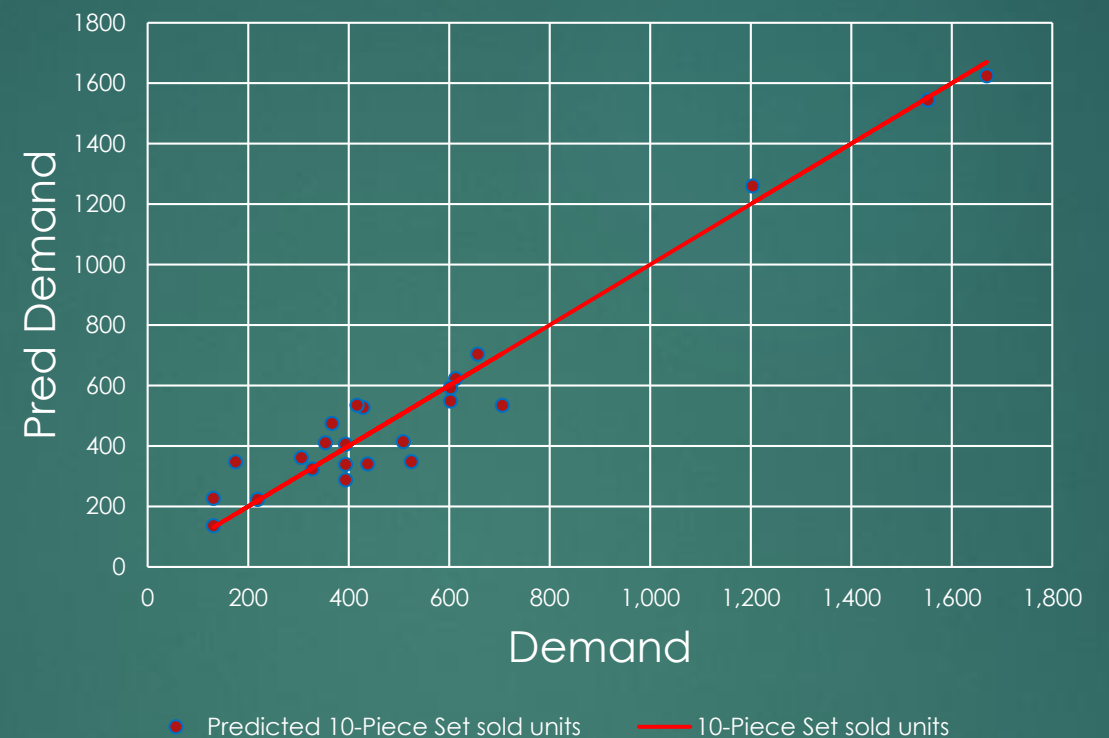
VIA EXCEL ANALYSIS TOOLPAK REGRESSION

5

Train Data– Train Data Predictions

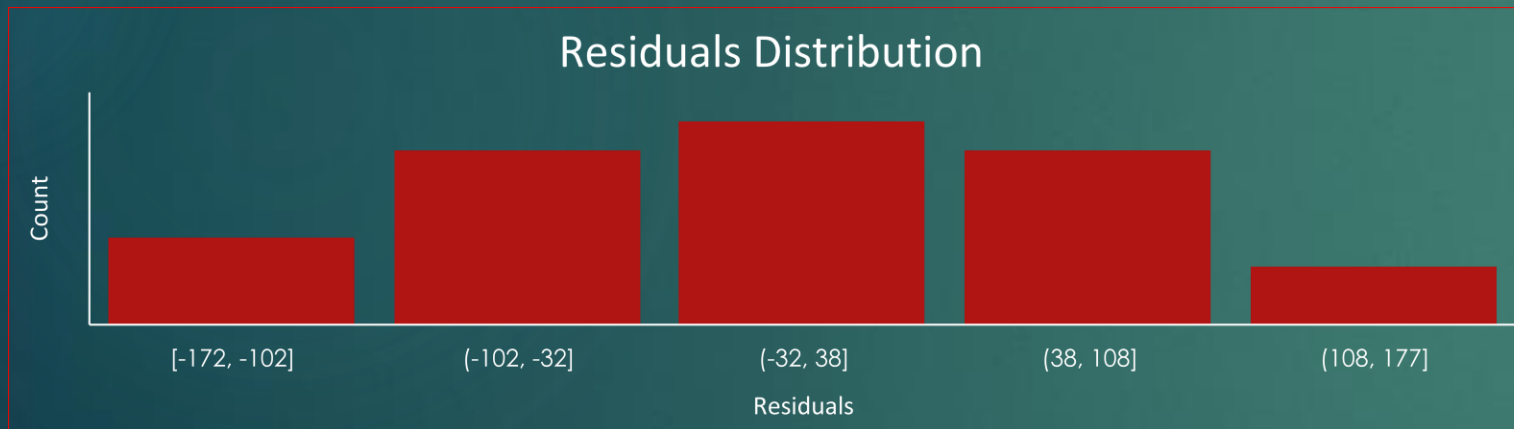
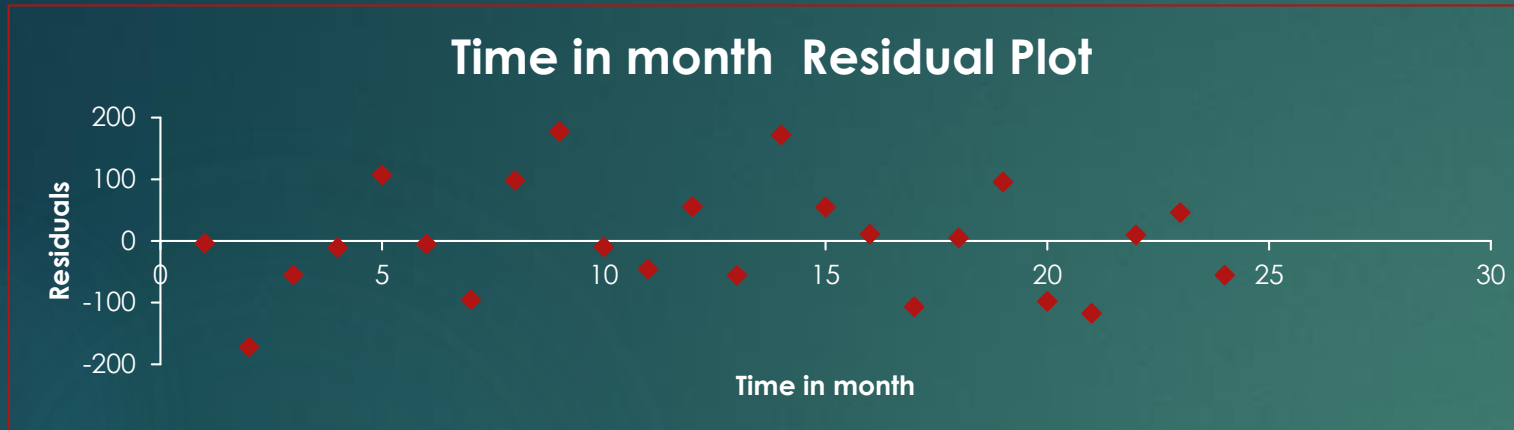


Train Data– Train Data Predictions



VIA EXCEL ANALYSIS TOOLPAK REGRESSION

6



Residual Validation

- Normality: YES
 - Skewness: 0.25
 - Kurtosis: - 0.29
- **Linearity: seems NOT PERFECT** → more positive values than negative ones
- Independence: YES
 - Durbin Watson p_value: 0.06
 - Between (- 0.25, 0.25)
- Equal Variance: seems OK.

Train Residual Mean: ~0

Test Residual Mean: -53

VIA EXCEL ANALYSIS TOOLPAK REGRESSION

7

TEST DATA

month year	10-Piece Set sold units	Time in month	10-Piece Set sale price	Feb	Mar	Apr	May	Jun	Jul	Aug	Oct	Nov	Dec
Jan-13	1069	25	\$799.95	0	0	0	0	0	0	0	0	0	0
Feb-13	1536	26	\$799.95	1	0	0	0	0	0	0	0	0	0
Mar-13	1490	27	\$799.95	0	1	0	0	0	0	0	0	0	0

TEST DATA PREDICTIONS

1331

1455

1469

METRICS

Mean Absolute Error	Weighted Mean Absolute Percentage Error
121.31	8.89%

VIA PYTHON, SCIKIT-LEARN AND
STATSMODELS LIBRARIES

VIA PYTHON SCIKIT-LEARN LIBRARY

9

Data Preparation

```
1 import pandas as pd
2 df = pd.read_csv("demand.csv")
3 df["10-Piece Set sale price"] = df["10-Piece Set sale price"].str.replace("$", "").astype("float")
4 df["10-Piece Set sold units"] = df["10-Piece Set sold units"].str.replace(",", "").astype("int")
5 df["dummy_1"] = pd.Series(data=[
6     -1, -1, 0, 0, 0, -1, -1, 0, 1, 1, 1, 0, -1, -1, 0, 0, 0, -1, -1, 0, 1, 1, 1, 0, -1, -1, 0
7 ])
```

Train Test Split

```
1 from sklearn.model_selection import train_test_split
2 x = df.drop(columns="10-Piece Set sold units")
3 y = df["10-Piece Set sold units"]
4 x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=3, shuffle=False)
```

Remarks

«dummy_1» variable added.

- This variable quantify peak, normal, valley seasons in 3 categories. For simplicity and accuracy, quantifications are determined from the first 12 months only.
- Shuffle=False as this is a time series data, we strictly want the very last rows to be in the test portion.

VIA PYTHON SCIKIT-LEARN LIBRARY

10

Pipeline

```
1 import numpy as np
2 from sklearn.model_selection import TimeSeriesSplit
3 from sklearn.pipeline import Pipeline
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.linear_model import LassoCV
6
7 alphas = np.logspace(-2, 2, 10)
8 cv = TimeSeriesSplit(n_splits=3, test_size=3)
9
10 input_steps=[
11     ("scaler", StandardScaler()),
12     ("model", LassoCV(alphas=alphas, cv=cv))
13 ]
14
15 pipe = Pipeline(input_steps)
```

Remarks

- **Imputer:** not used as growth in sales envisaged.
- **Polynomial featuring:** not used due to data scarcity and high dimensionality.
- **LinearRegression:** not favored as it causes overfitting on scarce data
- **StandardScaler:** used as value ranges differ between variables and LassoCV / RidgeCV uses regularization that stipulates scaling.
- **RidgeCV:** not favored as 'zero' coefficients envisaged for some categorical variables.
- **TimeSeriesSplit:** favored against LeaveOneOut as the latter generates convergence problems with LassoCV and data at hand is a real time series data.
- **SelectKBest:** not used as all categorical variables desired to be included at the beginning and conformity with excel solution.

VIA PYTHON SCIKIT-LEARN LIBRARY

11

Fit – Predict

```
1 pipe.fit(x_train, y_train)
2 pipe.score(x_train, y_train)
3
4 lasso = pipe.named_steps["model"]
5 coefficients = lasso.coef_
6 features = pipe.named_steps['scaler'].feature_names_in_
7
8 coef_df = pd.DataFrame({
9     'Variable': features,
10    'Coefficient': coefficients
11 })
12
13 print(coef_df.set_index("Variable", drop=True))
14
15 y_pred = pipe.predict(x_test)
16 y_pred_train = pipe.predict(x_train)
```

Remarks

- Train R square is 95.02%.
- Best alpha: 0.01, meaning the regularization is small.
- Nov, Oct → visible from coefficients that the demand is high.
- Price → price is more influential than Nov, Oct.

Variable	Coefficient
Time in month	102.731527
10-Piece Set sale price	-245.681800
Feb	38.436457
Mar	29.114520
Apr	37.173605
May	0.458259
Jun	-36.269958
Jul	-15.358889
Aug	2.801206
Oct	62.360829
Nov	80.507910
Dec	-15.228663
dummy_1	24.969061

VIA PYTHON SCIKIT-LEARN LIBRARY

12

Target and Pred Distribution: training data

```
1 ## Test and pred graphs
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4 sns.set_style("darkgrid")
5
6 fig, ax = plt.subplots(nrows=2, ncols=1, layout="constrained", sharex=True)
7
8 lines = [y_train, y_pred_train]
9 colors = ["red", "blue"]
10
11 for i in range(2):
12     sns.histplot(x=lines[i], ax=ax[0], lw=0, stat="percent",
13                 kde=True, kde_kws={"bw_adjust": 0.7, "cut": True, "cumulative": False},
14                 line_kws={"color": colors[i]}, alpha=0)
15     ax[0].set_ylim(0, 30)
16 ax[0].set_title("Training Target and Pred Distribution", fontweight="bold")
17
18 sns.scatterplot(x=y_train, y=y_pred_train, ax=ax[1])
19 sns.lineplot(x=y_train, y=y_train, color="red", ax=ax[1])
20
21 plt.savefig("train_target_pred_dist.png");
```



VIA PYTHON SCIKIT-LEARN LIBRARY

13

Residuals

```
1 pipe.score(x_test, y_test)
2
3 residuals_test = y_test - y_pred
4 residuals_test_mean = residuals_test.mean()
5 residuals_test_std = residuals_test.std()
6
7 residuals_train = y_train - y_pred_train
8 residuals_train_mean = residuals_train.mean()
9 residuals_train_std = residuals_train.std()
10
11 print("Residuals (test) mean:", np.round(residuals_test_mean, 2))
12 print("Residuals (test) std:", np.round(residuals_test_std, 2))
13 print("Residuals (train) mean:", np.round(residuals_train_mean, 2))
14 print("Residuals (train) std:", np.round(residuals_train_std, 2))
15 # Residuals (test) mean: -38.84
16 # Residuals (test) std: 166.17
17 # Residuals (train) mean: 0.0
18 # Residuals (train) std: 88.7
19
20 sns.residplot(x=y_pred_train, y=residuals_train, color="green", scatter_kws={"marker": "x"})
21 plt.title("Residuals", fontweight=650)
22 plt.savefig("residuals.png")
```

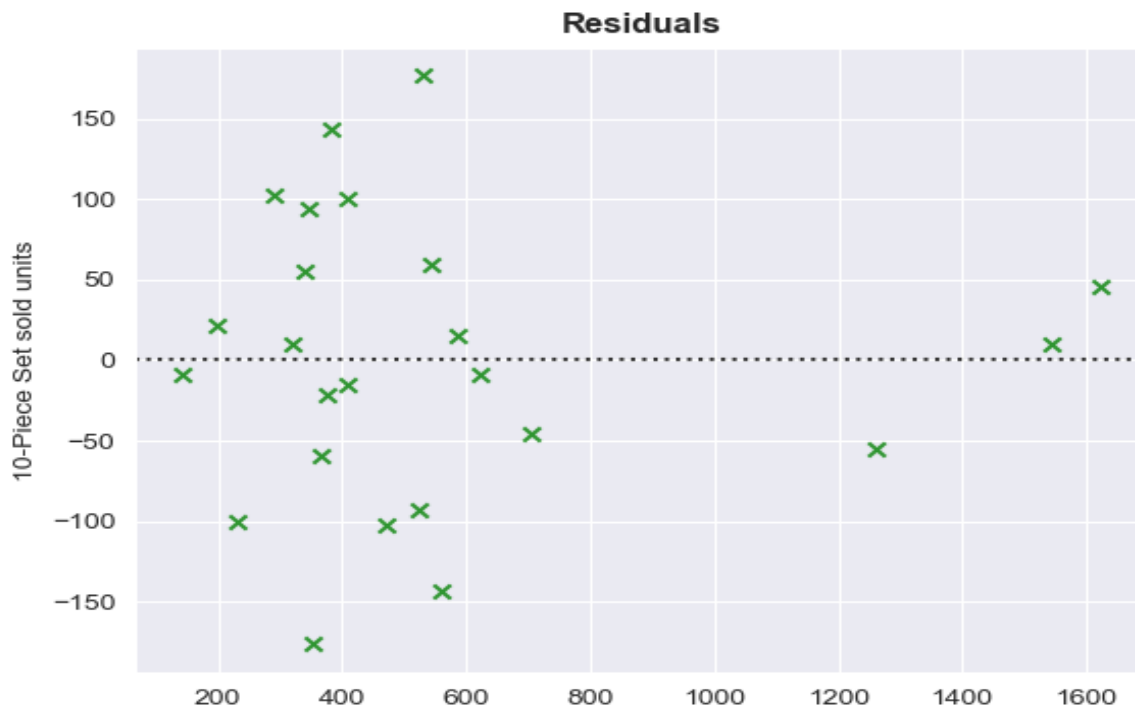
Remarks

- **Residuals (test) mean: problematic**, distant from 0.
But that's normal with scarce training and scarce test data.
- Residuals (train) mean: OK!
- **Test R square is 54.90%.**
 - Normal with scarce test data.

VIA PYTHON SCIKIT-LEARN LIBRARY

14

Residuals – validation -1



VIA PYTHON SCIKIT-LEARN LIBRARY

15

Residuals – Validation - 2



```
1 ## Residual validation
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4 sns.kdeplot(x=residuals_train, fill=True, color="red", bw_adjust=1, label="Test", alpha=0.6)
5 plt.title("Residuals (Train) Distribution", fontweight=650)
6 skewness = residuals_train.skew()
7 kurtosis = residuals_train.kurt()
8 print("Skewness is:", skewness)
9 # Skewness is: 2.7596181324392677e-05
10 print("Kurtosis is:", kurtosis)
11 # Kurtosis is: -0.3295592269287053
12 plt.savefig("residuals_train_distribution.png")
```

Remarks

- Normality:
 - Skewness: $2.75e-5$
 - Kurtosis: -0.33
 - Highly small values. OK!



VIA PYTHON SCIKIT-LEARN LIBRARY

16

Residuals – Validation - 3

```
1 import statsmodels.api as sm
2
3 x_temp = np.column_stack((y_pred_train, y_pred_train**2))
4 x_temp = sm.add_constant(x_temp)
5 model_temp = sm.OLS(residuals_train, x_temp).fit()
6 p_value = model_temp.pvalues.iloc[2]
7 print(f"p value (y_pred^2): {p_value:.4f}")
8 # p value (y_pred^2): 0.6156
9
10 if p_value < 0.05:
11     print("Model is not linear.
12           There is some non-linear relation between y_pred^2 and residuals.")
13 else:
14     print("Model is linear.")
15 # Model is linear.
```

Remarks

- Linearity:
 - OK!
 - Y_{pred}^2 p value: 0.6156
 - Square of target prediction values (training) does not pose a significant relation towards residuals

VIA PYTHON SCIKIT-LEARN LIBRARY

17

Residuals – Validation - 4

Remarks

```
1 from statsmodels.stats.stattools import durbin_watson
2 db = durbin_watson(residuals_train)
3 print("Durbin Watson is ", db)
4 # Durbin Watson is 1.84878921974064
5
6 p_value = 1 - db / 2
7 print("Independence p_value is", p_value)
8 # Independence p_value is 0.07560539012968004
9
10 if -0.25 < p_value < 0.25:
11     print("Model residuals are independent.")
12 else:
13     print("Model residuals are DEPENDENT.")
14 # Model residuals are independent.
```

- Independence:
 - OK!
 - Durbin Watson p value: 0.0756
 - NO orto-correlation.

VIA PYTHON SCIKIT-LEARN LIBRARY

18

Residuals – Validation - 5

Remarks

```
1 from statsmodels.stats.diagnostic import het_breuschpagan
2 x_train = sm.add_constant(x_train)
3 bp_test = het_breuschpagan(residuals_train, x_train)
4 p_value = bp_test[1]
5 print("Het Breuschpagan p value is", p_value)
6 # Het Breuschpagan p value is 0.03113018242218006
7
8 if p_value > 0.05:
9     print("Homoscedasticity, OK!")
10 else:
11     print("HETEROSCEDASTICITY, PROBLEM!, log conversion is needed.")
12 # HETEROSCEDASTICITY, PROBLEM!, log conversion is needed.
```

- Equal variance:
 - NOT SUPERB!
 - BP p value: 0.0311
 - HETEROSCEDASTICITY.
 - P value is not that small. No action needed.

VIA PYTHON SCIKIT-LEARN LIBRARY

19

TEST DATA

month year	10-Piece Set sold units	Time in month	10-Piece Set sale price	Feb	Mar	Apr	May	Jun	Jul	Aug	Oct	Nov	Dec
Jan-13	1069	25	\$799.95	0	0	0	0	0	0	0	0	0	0
Feb-13	1536	26	\$799.95	1	0	0	0	0	0	0	0	0	0
Mar-13	1490	27	\$799.95	0	1	0	0	0	0	0	0	0	0

TEST DATA PREDICTIONS

1297

1451

1465

METRICS

Mean Absolute Error	Weighted Mean Absolute Percentage Error
112.87	8.26%

COMPARISON SUMMARY

20

Metric	Metric-2	MS Excel	Python Scikit Learn	Winner
Train R Square		94.92%	95.02%	Python
Test R Square		42.93%	54.90%	Python
MAE		121.31	112.87	Python
WMAPE		8.89%	8.26%	Python
Train Residuals mean		~0	~0	-
Train Residuals Validation	Normality - Skewness	0.25	2.75E-5	Python
	Normality - Kurtosis	- 0.29	- 0.32	Excel
	Linearity	Seems NOT OK!	0.6156	Python
	Independence - DB	0.06	0.0756	Python
	Equal Variance	Seems OK!	0.0311	-
Test Residuals Mean		- 53	- 38	Python

- Test R square metrics are to grow with more data. Test size was ONLY 3.
- With the additional dummy categoric variable in Python, Python gained a slight edge over excel results